

2. Entorno de trabajo

Last edited time	October 14, 2026 8:55 PM
Practica	☐
Teoria	☑

Guía de estilo PEP8

- Reglas de PEP8 para IIC2233

Modularización

- Importación
  - Sentencias de importación
  - Variable `__name__`
- *Paths*
  - Extensiones de archivos
  - Problemas de portabilidad de *paths*
  - Navegación entre directorios
  - Pathlib

Terminal

Git

- Problemas comunes de Git
  - Repositorio des-sincronizado
  - Subir archivos muy pesados
  - Soluciones de problemas

Guía de estilo PEP8

▼ Ventajas de usar guías de estilo

- Código más legible
- Reparaciones (*debuggeo*) más rápidas
- Escribir más rápido nuestro código
- Nombres de archivos y variables predecibles

Reglas de PEP8 para IIC2233

▼ Aplicar `snake_case` para nombres de variables, atributos, funciones y métodos

- Se refiere a escribir palabras o frases compuestas usando guiones bajos como separadores.
- Todo minúsculas.

▼ Aplicar `CamelCase` para nombres de clases

- Se refiere a escribir palabras o frases compuestas sin separadores entre elementos (ni espacios) y cada elemento comienza con mayúscula.

- Nombres de variables descriptivos
- Importar módulos y librerías al comienzo del archivo
- Un espacio después de la coma
- Espacios a cada lado de operadores (`+` `-` `*` etc.)

- No usar tabs. Preferir 4 espacios para indentar.

▼ Líneas de largo máximo de 100 caracteres (incluyendo espacios e indentación)

- PEP8 permite hasta 79, pero el curso hasta 100.
- **Tips**
  - Ocupar variables auxiliares

  ▼ En casos extremos, ocupar `\` para separar una línea larga en la siguiente
    - No es necesario para que funcione, pero se prefiere encerrar entre paréntesis una expresión así y que la continuación empiece en el delimitador (paréntesis) inicial.

Modularización

- **Modularización**: consiste en separar el código de un programa en múltiples archivos.
- **Ventajas**
  - Mejor organización y legibilidad del código → mayor facilidad de debuggeo e inspección de código.
  - Facilita el reutilizar código.
  - Mejora proceso de conceptualización de flujo del programa

Importación

- Se pueda importar cualquier archivo `.py` con el comando `import` . Esto permite incorporar variables, funciones, clases y cualquier otro tipo de definición creada.

▼ Cuando Python llega a una línea con `import` , ejecuta todo el archivo, creando variables y definiciones. Luego, vuelve al archivo original y continua con las sentencias siguientes.

- Si hay una cadena de importaciones (A importa a B, en B se importa a C, etc), Python va a ejecutar todo el código del último módulo de la cadena y luego va marcha atrás para ejecutar el resto de los otros módulos.

▼ Python lleva registro de los módulos cuya importación ya se inició y no los vuelve a ejecutar.

- Esto evita un ciclo infinito de importaciones.
- Puede generar errores si un modulo que se está importando, requiere definiciones de otro módulo que aún no termina de importarse —>> mejor evitar los ciclos.

Sentencias de importación

▼ Importación completa

```
import modulo
```

Accede variables y otras usando `modulo.variable`

▼ Importación completa con alias

```
import modulo as alias
```

Accede elemento usando `alias.variable`

▼ Importación parcial

```
from modulo import variable, funcion, Clase
```

para acceder a los elementos se usa el nombre de estos `variable: MiClase = MiClase(<argumentos>)`

▼ Importación de un archivo dentro de una carpeta

```
import carpeta_con_modulo.modulo
import carpeta_con_modulo.modulo as alias_modulo
from carpeta_con_modulo import modulo
from carpeta_con_modulo.modulo import variable
```

- Si el módulo está en una subcarpeta:

  ```
  import carpeta.subcarpeta.modulo
  ```

⚠️ ☠️ Está **prohibido** ocupar `from modulo import *` ☠️

Variable `__name__`

- Permite revisar si un módulo está siendo ejecutado o está siendo importado.
  - Si está siendo ejecutado toma el valor de `"__main__"`
  - Si está siendo importado toma el valor de un string con el nombre del módulo (ej. `"modulo"`), independientemente si fue importado bajo un alias.

Paths

El módulo `os` de Python provee una interfaz portable para ejecutar operaciones relacionadas al SO y al sistema de archivos.

- Un ***path* (ruta)** representa una ubicación en un sistema de archivos.
- Pueden ser de dos tipos

▼ **Path absoluto**

- Dirección de archivo o directorio desde la **raíz** del sistema de archivos.
  - Usualmente es `\` , pero hay otros (ej. Windows usa `C:\`)

▼ No varia con respecto al directorio donde se ejecuta el programa

- PERO requiere que la ruta exista exactamente igual en todos los sistemas de archivos donde se ejecute el programa.

▼ **Path relativo**

- Dirección relativa a cierto directorio.
- Más simple, pero requiere más precaución cuando se ejecuta el programa en un directorio distinto al esperado.

⚠️ Solo ocupar **rutas relativas** en el curso

- Un path se divide en dos partes
  - `dirname` (nombre del directorio), carpeta donde está el archivo o directorio objetivo.
    - `os.path.dirname(<path>)` retorna un str con el `dirname`
  - `basename` (nombre de archivo o directorio objetivo)
    - `os.path.basename(<path>)` retorna un str con el `basename`

Extensiones de archivos

- Sirven para darle una **pista** al usuario sobre el tipo de archivo y al sistema operativo para saber con qué programa leer el archivo.

▼ Se puede ocupar la función `os.path.splitext(<basename>)` para separar la extensión del resto del nombre del archivo.

```
nombre_sin_extension, extension = os.path.splitext(basename1)
print(nombre_sin_extension)
print(extension)
```

- Ojo, una extensión informa del tipo de archivo, pero **no determina el tipo del archivo.**

- Es decir, si uno cambia la extensión de un archivo, por ejemplo de `.txt` a `.jpg` , todavía puedes abrir el archivo como si fuera de texto.

Problemas de portabilidad de *paths*

- Las rutas en Windows ocupan `\` como separador, pero también aceptan `/` (que se usan en Linux y macOS).

▼ El modulo `os.path` provee funcionalidades para reescribir rutas usando los **caracteres de separación nativos** del SO donde se ejecute el programa.

```
>>> ruta = os.path.join("home", "pedro", "Libros", "python.pdf")
>>> ruta
'home/pedro/Libros/python.pdf'
```

`os.path.join` **forma una ruta usando una serie de argumentos en formato string que corresponden a los directorios y archivo objetivo.**

- Se recomienda ocupar paths de la segunda forma, para asegurar portabilidad de nuestros programas.

Navegación entre directorios

- `os.listdir(<ruta>)` retorna una lista con los nombres de directorios y archivos que se encuentran directamente dentro de la `<ruta>` de ese directorio.
- `os.walk()` es una función que genera los nombres de los archivos y directorios que se encuentran en un árbol de directorios.
  - Genera un tuple de 3 elementos (**raiz, directorios, archivos**) para cada directorio que encuentra en el árbol.

  ▼ Notación

  ```
  for raiz, directorios, archivos in os.walk("data", topdown=True):
      print("Raíz:", raiz)
      print()
      print("Archivos:")
      for archivo in archivos:
          print(os.path.join(raiz, archivo))
      print()
      print("Directorios:")
      for directorio in directorios:
          print(os.path.join(raiz, directorio))
      print("-" * 30)
  ```

  - El parámetro `topdown` permite decidir si la navegación será desde el directorio raíz (`topdown=True`) o desde sus elementos hojas (`topdown=False`).
- `os.getcwd()` retorna un string con el directorio donde se está ejecutando el archivo.

Pathlib

- Módulo de Python que ofrece **clases** capaces de representar las rutas del sistema de archivos.

▼ Instanciar un objeto con clase `Path`

`path = Path(<ruta>)`

▼ Métodos y atributos de la clase `Path`

`path` se refiere a una instancia de clase

- `path.name` retorna el nombre
- `path.suffix` retorna la extensión

- `path.is_dir()` y `path.is_file()` retornan True si la ruta corresponde a un directorio o un archivo, respectivamente.
- `path.absolute()` retorna la ruta absoluta
- `path.exists()` retorna True si el archivo existe en la ruta
- `path.read_text()` retorna el contenido de un archivo de texto
- `path.iterdir()` retorna un iterador con un objeto `Path` de cada archivo o subdirectorio dentro de la ruta.
- `path.cwd()` retorna un objeto `Path` del directorio donde se ejecutó el archivo

▼ El objeto instanciado de la clase Path también permite lectura y escritura mediante su método `open(mode="r")`

- Luego se puede ocupar los métodos de `write` y `read`.

Terminal

▼ Principales ventajas

- Múltiples terminales abiertas y cada una para diferentes propósitos (ej. ejecutar diferentes archivos Python al mismo tiempo).
  - Las IDEs viene predeterminadas para ejecutar 1 archivo a la vez.
- Conocer el working directory a todo momento
  - Evitar problemas con la apertura de archivos, imports y rutas relativas.

▼ Principales comandos

- `cd` (sin argumentos) imprime el directorio actual
- `cd` cambia de directorios
  - `..` para subir un directorio
- `dir` lista los archivos/carpetas en el directorio
- `mkdir` crea una nueva carpeta con el nombre que se le pase
  - Alternativamente, también recibe una ruta con la dirección de la nueva carpeta
    `mkdir C:\Users\Nico\Documents\carpeta_nueva`
- `echo` permite mostrar un mensaje de salida en la terminal (similar a un `print` de Python en consola)
  - Lo usamos para redirigir esa salida a un archivo:
    `echo "Texto" > nombre_archivo`
  - Si uno se le pasa `""` en lugar de un texto, el archivo se crea con estas comillas. En caso de necesitar un archivo completamente vacio, se puede ocupar el siguiente comando de **cmd:**
    `type nul > nuevo.py`
- `python`
  - `python archivo.py` ejecuta este archivo que está en el directorio actual
  - `python <ruta>` ejecuta el archivo que esté en esa ruta.
    - En este caso se ejecuta desde la ruta actual, no desde la carpeta donde se encuentra el archivo ⚠️ ⚠️

▼ Formato de un comando

`comando argumentos* -opciones*`

- `comando` el nombre del comando

▼ Algunos comandos actúan de forman distinta según el número de `argumentos` que reciben

- En Windows, `cd` sin argumentos muestra el directorio actual.
- Si uno pone `python3` va a abrirse la consola, pero si uno añade una archivo, por ejemplo `python3 hola.py` , entonces ejecuta el archivo.

- `pip list` muestra las librerias instaladas
  vs `pip install nombre_libreria` instala una librería en específico

▼ `-opciones` (o *flags*) especifica opciones durante la ejecución del comando

- Cada opción se escribe como `-opcion parametro*` , donde el parámetro puede ser opcional según la opción que usemos.
- ej. `ls -a` muestra todos los archivos
  `pip install pyqt6 -q` instala `pyqt6` de forma silenciosa (sin imprimir tantas cosas en consola)

Git

- Sistema distribuido de control de versiones

▼ Repositorios

Almacenamiento virtual que guarda el estado final de los archivos y todas las diferentes versiones que se han ido registrando

▼ Comandos básicos

- `git clone <url>` permite **clonar** un repositorio al directorio actual. Recibe la URL a un repositorio (`.git`)
- `git pull` permite **actualizar** el repositorio local con los contenidos del repositorio remoto
- `git add` recibe un nombre de archivo (o su path) y permite pasarlo desde nuestro computador a la lista de cambios (o ***staging area***)
- `git commit` crea una versión de los archivos.
  - Se ocupa con `git commit -m "mensaje descriptivo"`
  - Los archivos en la lista de cambios pasan al **repositorio local.**
  - Se registra la hora del commit y los cambios y archivos que se incluyeron con dicho commit.
- `git push` sube las versiones que estén en el repositorio local al repositorio remoto.

⚠️

- Cada commit guarda una foto (**snapshot**) **completa** del proyecto en ese momento. En otras palabras, incluye todos los archivos que existían al hacer el commit, no solo los que se agregaron con este.
  - Se registra un punto en la evolución del proyecto.
- Los commits se guardan uno dentro del otro, pero **no son acumulativos**. Es decir:
  - Si uno borra un archivo, la nueva snapshot no la incluye.
  - Puedes recuperar versiones previas de archivos (o incluso los archivos mismos).

Problemas comunes de Git

Repositorio des-sincronizado

- A veces el repositorio local puede estar desincronizado con respecto al repositorio online, porque puede estar siendo trabajado desde varios pcs.
  `"you may want to first integrate the remote changes before pushing again"`
- Si uno trata de hacer `git pull` podría fallar y pedirte decidir cómo reconciliar las dos versiones
  `You have divergent branches and need to specify how to reconcile them`
  - Para ocupar la estrategia de **merge:**

```
git config --global pull.rebase false
git pull
git push
```

- Puede abrirse un **editor de texto** para el commit de combinación, pero basta con guardar y salir.

- Si las versiones del repositorio editaron archivos distintos, con lo anterior todo queda sincronizado. En cambio, si editaron el mismo archivo, aparecerá un error llamado **merge conflict.**
    - Evitar solucionarlo desde la terminal o siguiendo tutoriales online. Preferir hacerlo desde la IDE.

Subir archivos muy pesados

- Git no tiene límite de tamaño, pero GitHub acepta archivos hasta 100 MB.
- Si uno trata de hacer `git push` con un archivo de mayor tamaño, va a generarse un error. Luego, aunque uno elimine el archivo, la próxima vez que haga `git push` también nos va a lanzar error (porque queda el historial de que estuvo ese archivo)

Soluciones de problemas

Para la mayoría de problemas, se puede crear otra copia B del repositorio con `git clone` y luego pasarle las carpetas y archivos que se deseen mantener desde la copia A (donde estaba el problema). Luego, hacer `git add` , `git commit` y `git push` para quedarnos con esa versión del proyecto, y eliminar la copia A.